```
This program will take several numbers then average them
How many numbers would you like to average: 3
Number 1
Enter a number: 1
Number 2
Enter a number: 4
Number 3
Enter a number: 3
The average was: 2.66666666667
```

6.0.3 Exercises

Write a program that asks the user their name, if they enter your name say "That is a nice name", if they enter "John Cleese" or "Michael Palin", tell them how you feel about them ;), otherwise tell them "You have a nice name."

Solution

```
name = input('Your name: ')
if name == 'Bryn':
    print('That is a nice name.')
elif name == 'John Cleese':
    print('... some funny text.')
elif name == 'Michael Palin':
    print('... some funny text.')
else:
    print('You have a nice name.')
```

Modify the higher or lower program from this section to keep track of how many times the user has entered the wrong number. If it is more than 3 times, print "That must have been complicated." at the end, otherwise print "Good job!"

Solution

```
number = 7
guess = -1
count = 0

print("Guess the number!")
while guess != number:
    guess = int(input("Is it... "))
    count = count + 1
    if guess == number:
        print("Hooray! You guessed it right!")
    elif guess < number:
        print("It's bigger...")
    elif guess > number:
        print("It's not so big.")

if count > 3:
    print("That must have been complicated.")
else:
    print("Good job!")
```

Write a program that asks for two numbers. If the sum of the numbers is greater than 100, print ”That is a big number.”

Solution

```
number1 = float(input('1st number: '))
number2 = float(input('2nd number: '))
if number1 + number2 > 100:
    print('That is a big number.')
```

7 Debugging

7.0.1 What is debugging?

> "As soon as we started programming, we found to our surprise that it wasn't as easy to get programs right as we had thought. Debugging had to be discovered. I can remember the exact instant when I realized that a large part of my life from then on was going to be spent in finding mistakes in my own programs." — *Maurice Wilkes discovers debugging* , 1949

By now if you have been messing around with the programs you have probably found that sometimes the program does something you didn't want it to do. This is fairly common. Debugging is the process of figuring out what the computer is doing and then getting it to do what you want it to do. This can be tricky. I once spent nearly a week tracking down and fixing a bug that was caused by someone putting an `x` where a `y` should have been.

This chapter will be more abstract than previous chapters.

7.0.2 What should the program do?

The first thing to do (this sounds obvious) is to figure out what the program should be doing if it is running correctly. Come up with some test cases and see what happens. For example, let's say I have a program to compute the perimeter of a rectangle (the sum of the length of all the edges). I have the following test cases:

height	**width**	**perimeter**
3	4	14
2	3	10
4	4	16
2	2	8
5	1	12

I now run my program on all of the test cases and see if the program does what I expect it to do. If it doesn't then I need to find out what the computer is doing.

More commonly some of the test cases will work and some will not. If that is the case you should try and figure out what the working ones have in common. For example here is the output for a perimeter program (you get to see the code in a minute):

```
Height: 3
Width: 4
perimeter = 15
```

```
Height: 2
Width: 3
perimeter = 11
```

```
Height: 4
Width: 4
perimeter = 16
```

```
Height: 2
Width: 2
perimeter = 8
```

```
Height: 5
Width: 1
perimeter = 8
```

Notice that it didn't work for the first two inputs, it worked for the next two and it didn't work on the last one. Try and figure out what is in common with the working ones. Once you have some idea what the problem is finding the cause is easier. With your own programs you should try more test cases if you need them.

7.0.3 What does the program do?

The next thing to do is to look at the source code. One of the most important things to do while programming is reading source code. The primary way to do this is code walkthroughs.

A code walkthrough starts at the first line, and works its way down until the program is done. `while` loops and `if` statements mean that some lines may never be run and some lines are run many times. At each line you figure out what Python has done.

Lets start with the simple perimeter program. Don't type it in, you are going to read it, not run it. The source code is:

```
height = int(input("Height: "))
width = int(input("Width: "))
print("perimeter =", width + height + width + width)
```

Question: What is the first line Python runs?

Answer: The first line is always run first. In this case it is: `height = int(input("Height: "))`

What does that line do?

Prints `Height:`, waits for the user to type a string in, and then converts the string to an integer variable height.

What is the next line that runs?

In general, it is the next line down which is: `width = int(input("Width: "))`

What does that line do?

Prints `Width:`, waits for the user to type a number in, and puts what the user types in the variable width.

What is the next line that runs?

When the next line is not indented more or less than the current line, it is the line right afterwards, so it is: `print("perimeter = ", width + height + width + width)` (It may also run a function in the current line, but that's a future chapter.)

What does that line do?

First it prints `perimeter =`, then it prints the sum of the values contained within the variables, `width` and `height`, from `width + height + width + width`.

Does `width + height + width + width` calculate the perimeter properly?

Let's see, perimeter of a rectangle is the bottom (width) plus the left side (height) plus the top (width) plus the right side (huh?). The last item should be the right side's length, or the height.

Do you understand why some of the times the perimeter was calculated "correctly"?

It was calculated correctly when the width and the height were equal.

The next program we will do a code walkthrough for is a program that is supposed to print out 5 dots on the screen. However, this is what the program is outputting:

```
. . . .
```

And here is the program:

```
number = 5
while number > 1:
    print(".",end=" ")
    number = number - 1
print()
```

This program will be more complex to walkthrough since it now has indented portions (or control structures). Let us begin.

What is the first line to be run?

The first line of the file: `number = 5`

What does it do?

Puts the number 5 in the variable number.

What is the next line?

The next line is: `while number > 1:`

What does it do?

Well, `while` statements in general look at their expression, and if it is true they do the next indented block of code, otherwise they skip the next indented block of code.

So what does it do right now?

If `number > 1` is true then the next two lines will be run.

So is `number > 1`?

The last value put into `number` was `5` and `5 > 1` so yes.

So what is the next line?

Since the `while`was true the next line is: `print(".",end=" ")`

What does that line do?

Prints one dot and since the extra argument `end=" "` exists the next printed text will not be on a different screen line.

What is the next line?

`number = number - 1` since that is following line and there are no indent changes.

What does it do?

It calculates `number - 1`, which is the current value of `number` (or 5) subtracts 1 from it, and makes that the new value of number. So basically it changes `number`'s value from 5 to 4.

What is the next line?

Well, the indent level decreases so we have to look at what type of control structure it is. It is a `while` loop, so we have to go back to the `while` clause which is `while number > 1:`

What does it do?

It looks at the value of number, which is 4, and compares it to 1 and since `4 > 1` the while loop continues.

What is the next line?

Since the while loop was true, the next line is: `print(".",end=" ")`

What does it do?

It prints a second dot on the line, ending by a space.

What is the next line?

No indent change so it is: `number = number - 1`

And what does it do?

It takes the current value of number (4), subtracts 1 from it, which gives it 3 and then finally makes 3 the new value of number.

What is the next line?

Since there is an indent change caused by the end of the while loop, the next line is: `while number > 1:`

What does it do?